

Documentation

Chart types

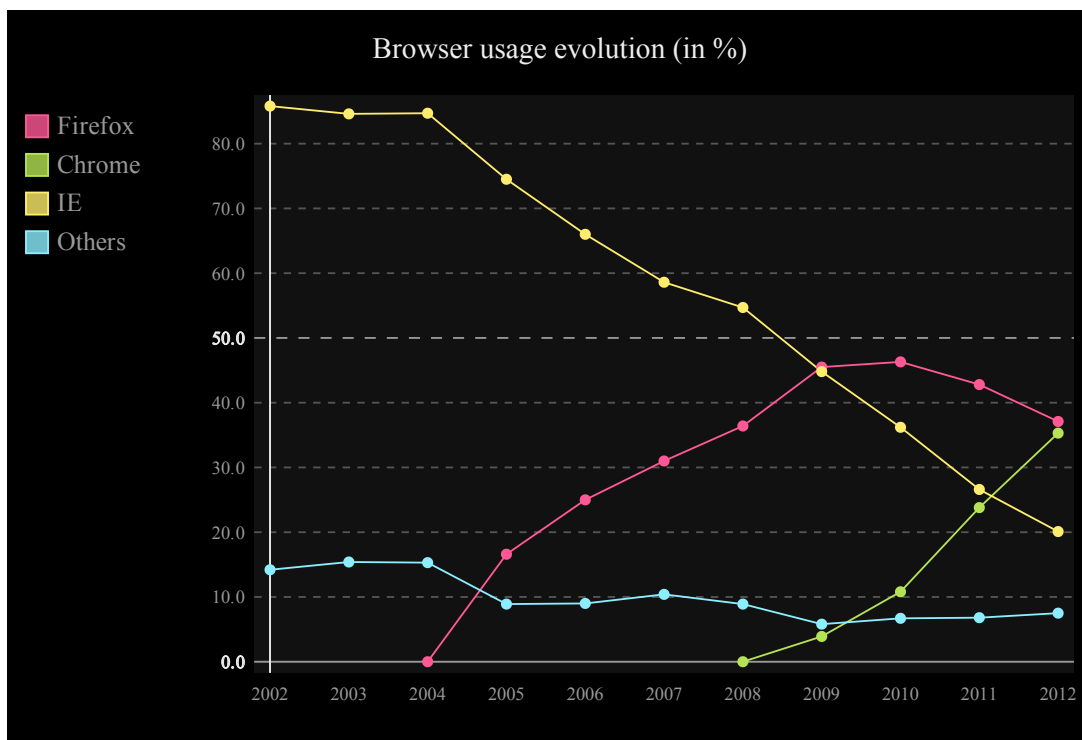
pygal provides 10 kinds of charts:

- Line charts
 - Basic
 - Stacked
- Bar charts / Histograms
 - Basic
 - Stacked
 - Horizontal
- XY charts
 - Basic
 - Scatter Plot
 - DateY
- Pies
 - Basic
 - Multi-series pie
- Radar charts
 - Basic
- Box plot
 - Basic
- Dot charts
 - Basic
- Funnel charts
 - Basic
- Gauge charts
 - Basic
- Pyramid charts
 - Basic
- Worldmap charts
 - Basic
- Country charts
 - French map

Line charts

Basic

Basic simple line graph:



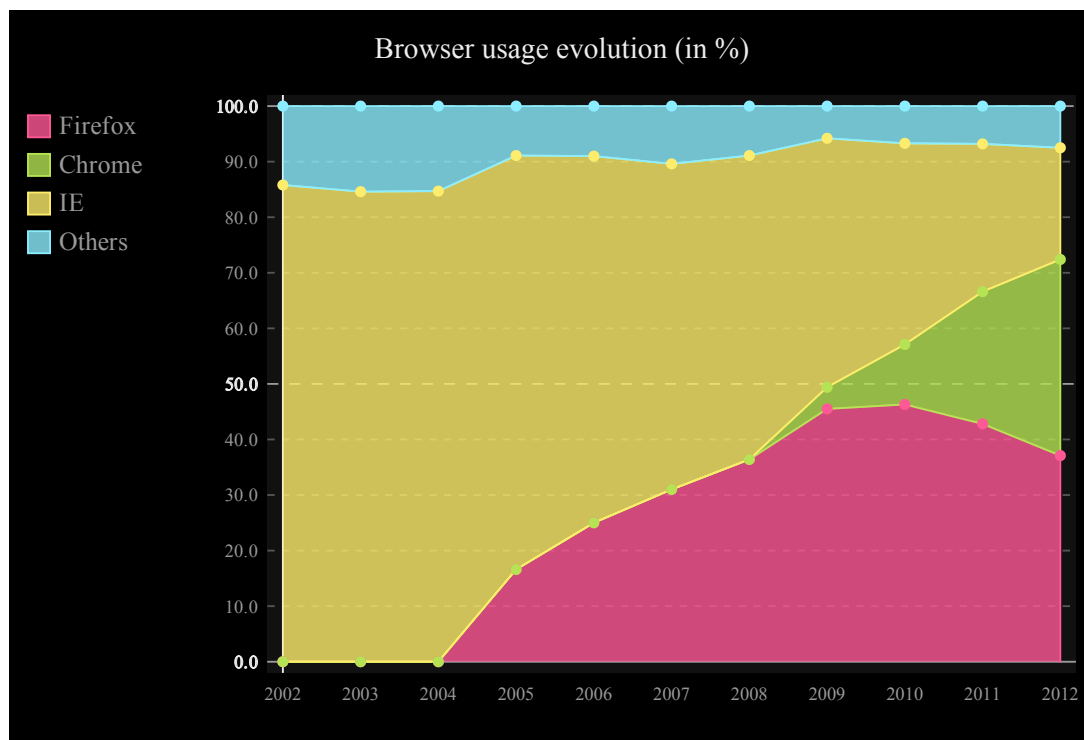
```

line_chart = pygal.Line()
line_chart.title = 'Browser usage evolution (in %)'
line_chart.x_labels = map(str, range(2002, 2013))
line_chart.add('Firefox', [None, None, 0, 16.6, 25, 31, 36, 45, 46, 43, 37])
line_chart.add('Chrome', [None, None, None, None, None, None, None, 0, 4, 11, 24, 35])
line_chart.add('IE', [85.8, 84.6, 84.7, 74.5, 66, 59, 55, 45, 36, 27, 20])
line_chart.add('Others', [14.2, 15.4, 15.3, 8.9, 9, 10, 9, 6, 7, 7, 7])
line_chart.render()

```

Stacked

Same graph but with stacked values and filled rendering:



```

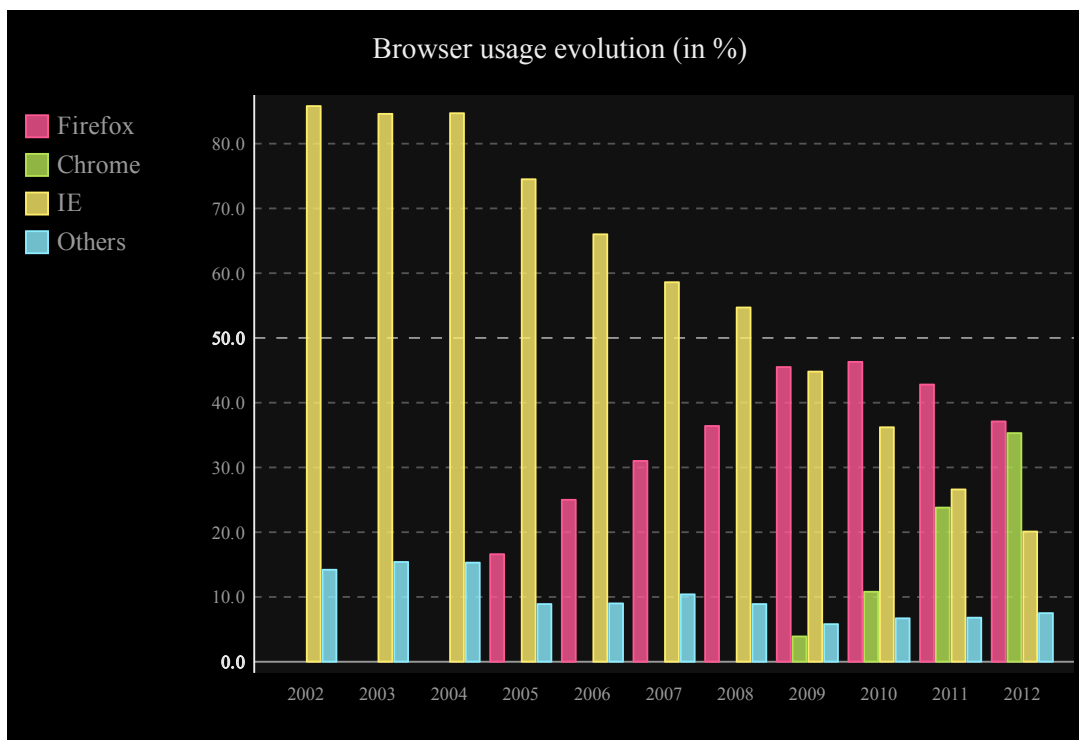
stackedline_chart = pygal.StackedLine(fill=True)
stackedline_chart.title = 'Browser usage evolution (in %)'
stackedline_chart.x_labels = map(str, range(2002, 2013))
stackedline_chart.add('Firefox', [None, None, 0, 16.6, 25.0, 30.0, 36.0, 45.0, 46.0, 43.0, 37.0])
stackedline_chart.add('Chrome', [None, None, None, None, 0.0, 0.0, 0.0, 30.0, 30.0, 30.0, 30.0])
stackedline_chart.add('IE', [85.8, 84.6, 84.7, 74.5, 74.5, 74.5, 74.5, 74.5, 74.5, 74.5, 74.5])
stackedline_chart.add('Others', [14.2, 15.4, 15.3, 8.9, 8.9, 8.9, 8.9, 8.9, 8.9, 8.9, 8.9])
stackedline_chart.render()

```

Bar charts / Histograms

Basic

Basic simple bar graph:



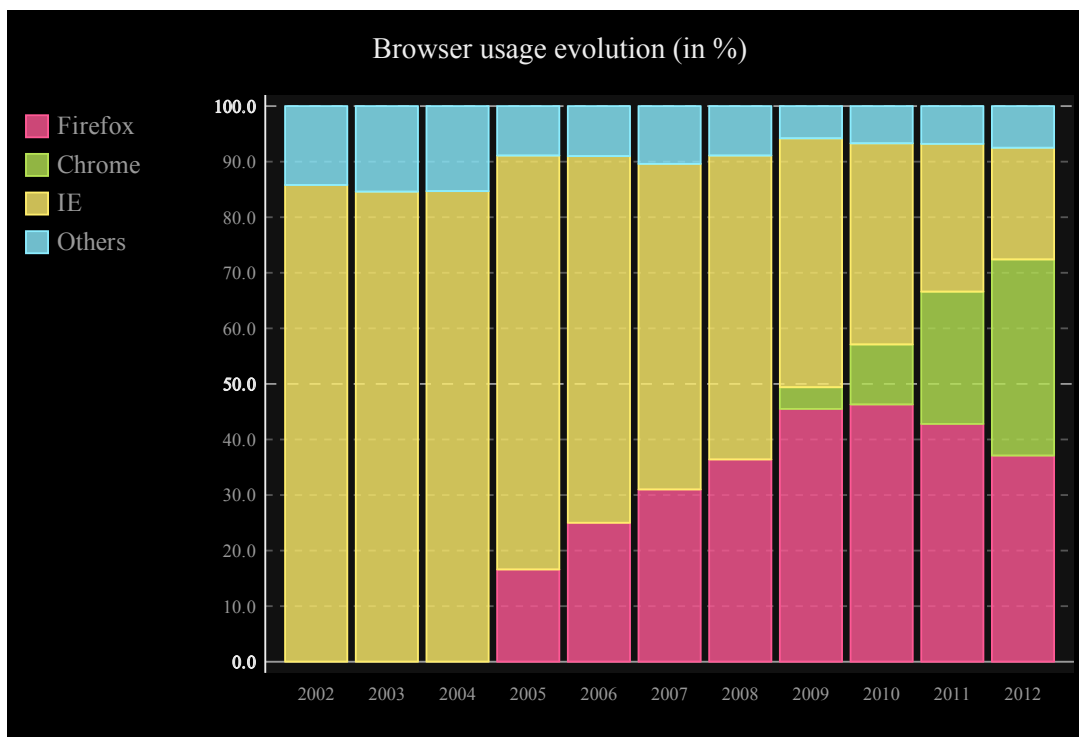
```

bar_chart = pygal.Bar()
bar_chart.title = 'Browser usage evolution (in %)'
bar_chart.x_labels = map(str, range(2002, 2013))
bar_chart.add('Firefox', [None, None, 0, 16.6, 25, 31, 45, 46, 43, 37, 37])
bar_chart.add('Chrome', [None, None, None, None, None, None, None, None, 11, 24, 35, 35])
bar_chart.add('IE', [85.8, 84.6, 84.7, 74.5, 66, 58, 45, 45, 36, 27, 20, 20])
bar_chart.add('Others', [14.2, 15.4, 15.3, 8.9, 9, 10, 10, 10, 7, 6, 7, 7])
bar_chart.render()

```

Stacked

Same graph but with stacked values:



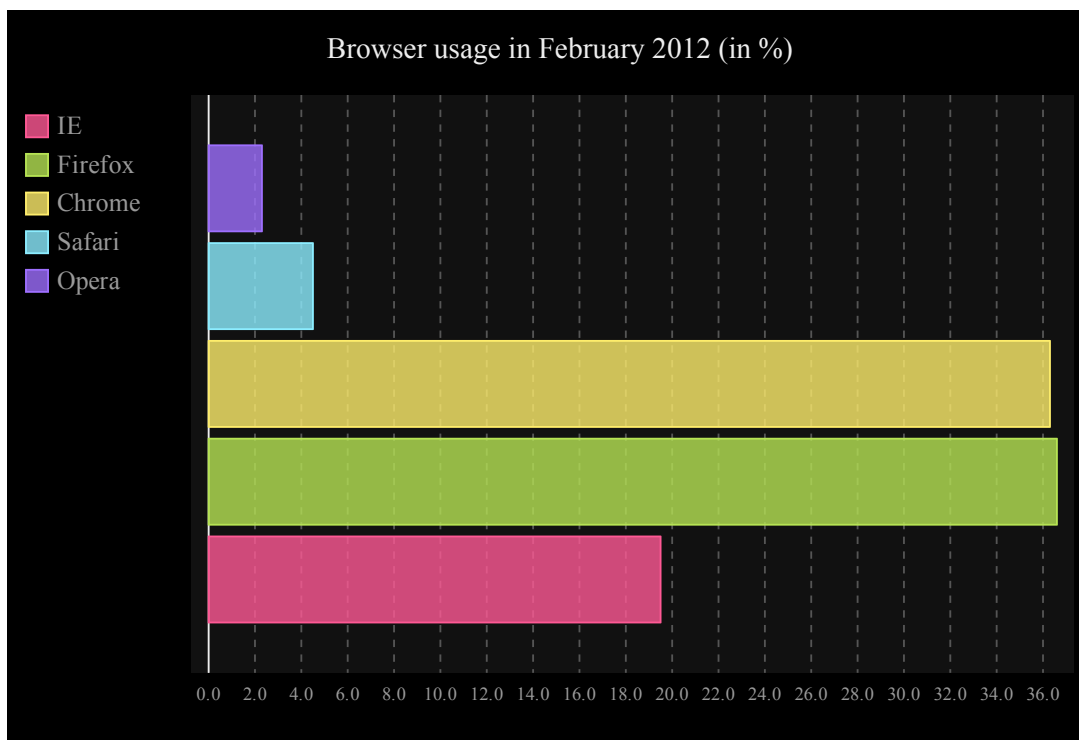
```

stackedbar_chart = pygal.StackedBar()
stackedbar_chart.title = 'Browser usage evolution (in %)'
stackedbar_chart.x_labels = map(str, range(2002, 2013))
stackedbar_chart.add('Firefox', [None, None, 0, 16.6, 25.0, 31.0, 36.0, 45.0, 46.0, 43.0, 37.0])
stackedbar_chart.add('Chrome', [None, None, None, None, 0, 0, 0, 5.0, 12.0, 24.0, 35.0])
stackedbar_chart.add('IE', [85.8, 84.6, 84.7, 74.5, 74.5, 69.0, 64.0, 50.0, 42.0, 33.0, 28.0])
stackedbar_chart.add('Others', [14.2, 15.4, 15.3, 8.9, 0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])
stackedbar_chart.render()

```

Horizontal

Horizontal bar diagram:

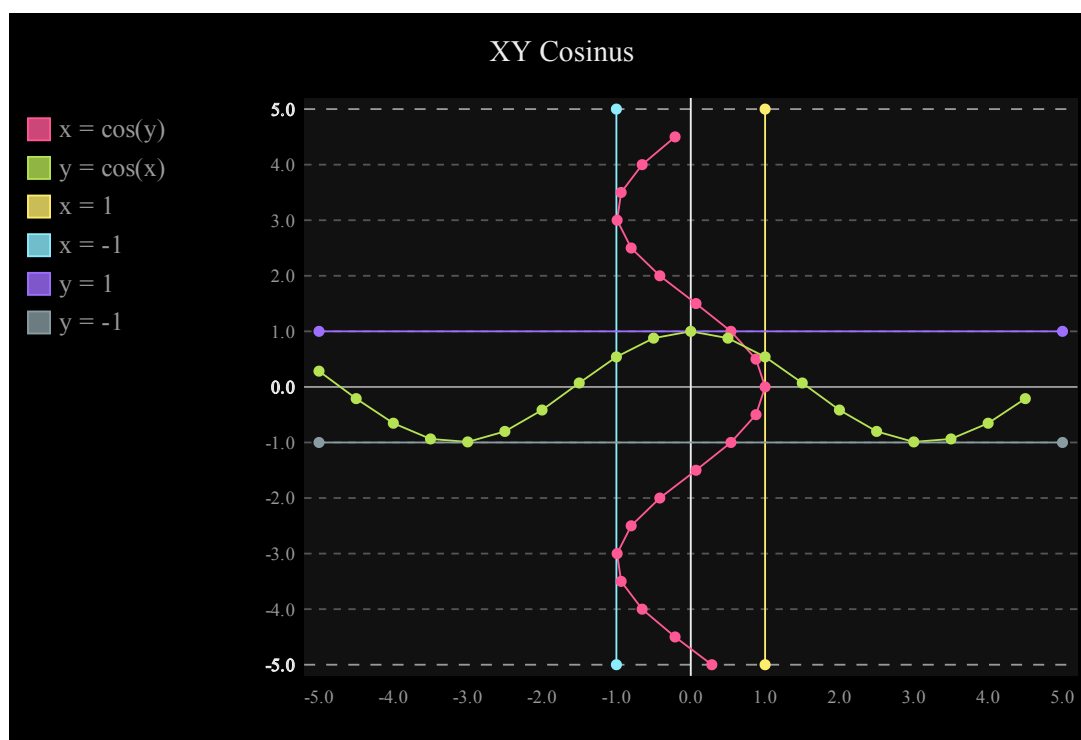


```
horizontalbar_chart = pygal.HorizontalBar()
horizontalbar_chart.title = 'Browser usage in February 2012'
horizontalbar_chart.add('IE', 19.5)
horizontalbar_chart.add('Firefox', 36.6)
horizontalbar_chart.add('Chrome', 36.3)
horizontalbar_chart.add('Safari', 4.5)
horizontalbar_chart.add('Opera', 2.3)
horizontalbar_chart.render()
```

XY charts

Basic

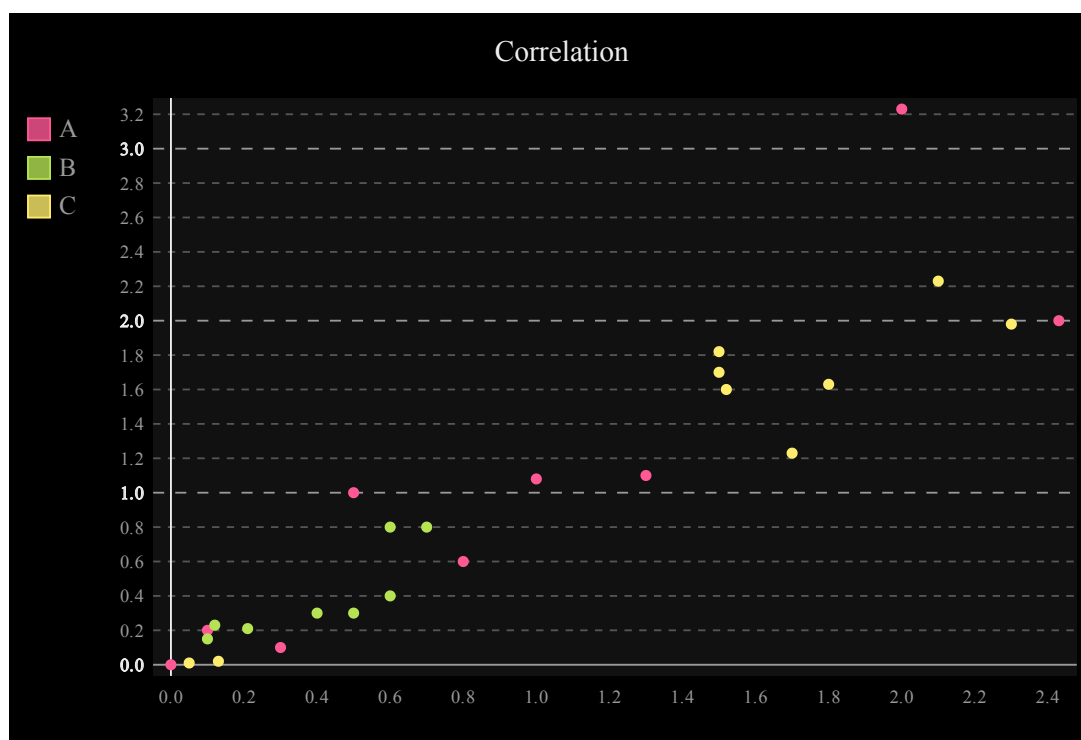
Basic XY lines, drawing cosinus:



```
from math import cos
xy_chart = pygal.XY()
xy_chart.title = 'XY Cosinus'
xy_chart.add('x = cos(y)', [(cos(x / 10.), x / 10.) for x
xy_chart.add('y = cos(x)', [(x / 10., cos(x / 10.)) for x
xy_chart.add('x = 1', [(1, -5), (1, 5)])
xy_chart.add('x = -1', [(-1, -5), (-1, 5)])
xy_chart.add('y = 1', [(-5, 1), (5, 1)])
xy_chart.add('y = -1', [(-5, -1), (5, -1)])
xy_chart.render()
```

Scatter Plot

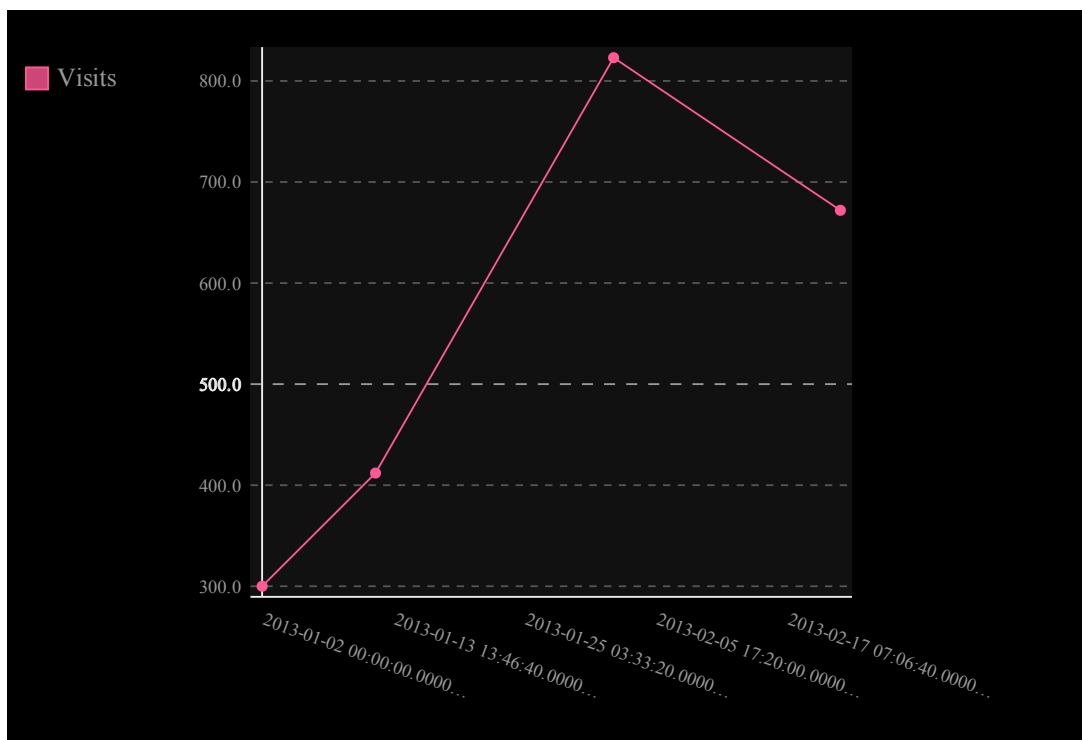
Disabling stroke make a good scatter plot



```
xy_chart = pygal.XY(stroke=False)
xy_chart.title = 'Correlation'
xy_chart.add('A', [(0, 0), (.1, .2), (.3, .1), (.5, 1), (.8, .6), (1.0, 1.1), (1.3, 1.1), (2.0, 3.2), (2.4, 2.0)])
xy_chart.add('B', [(0.1, .15), (.12, .23), (.4, .3), (.6, .4), (.7, .8), (.8, .8)])
xy_chart.add('C', [(0.05, .01), (.13, .02), (1.5, 1.7), (1.5, 1.8), (1.5, 1.6), (1.7, 1.2), (1.8, 1.6), (2.1, 2.2), (2.3, 2.0)])
xy_chart.render()
```

DateY

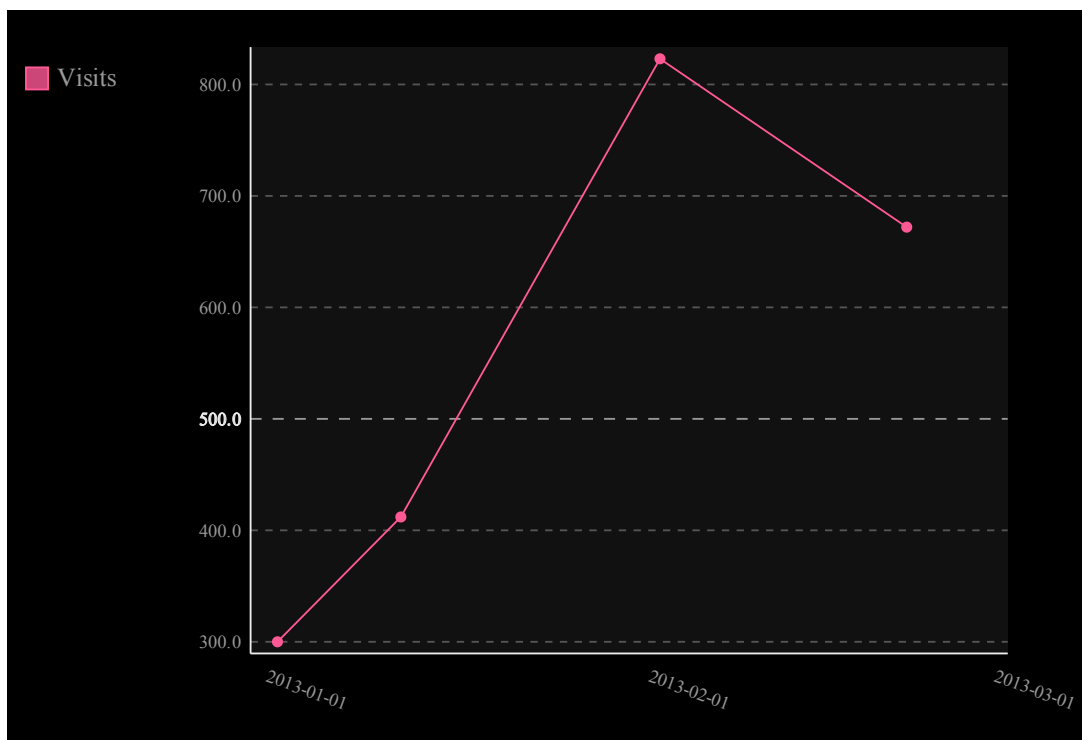
You can index values by dates (Thanks to [Snarkturne](#))



```
from datetime import datetime, timedelta
datey = pygal.DateY(x_label_rotation=20)
datey.add("Visits", [
    (datetime(2013, 1, 2), 300),
    (datetime(2013, 1, 12), 412),
    (datetime(2013, 2, 2), 823),
    (datetime(2013, 2, 22), 672)
])
datey.render()
```

The x axis and tool tip x labels can be specified using `x_label_format`. This uses the formatting string for strftime from [here](#)

The x labels can also be specified, using an array of datetime objects.

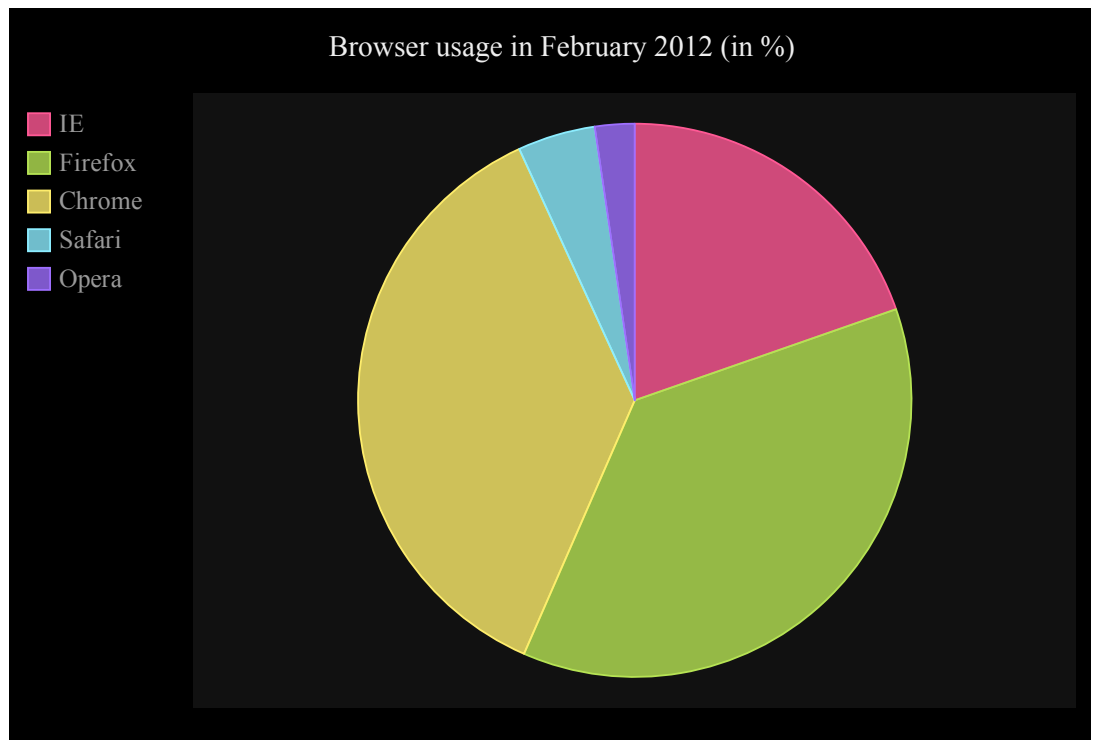


```
from datetime import datetime, timedelta
datey = pygal.DateY(x_label_rotation=20)
datey.add("Visits", [
    (datetime(2013, 1, 2), 300),
    (datetime(2013, 1, 12), 412),
    (datetime(2013, 2, 2), 823),
    (datetime(2013, 2, 22), 672)
])
datey.x_label_format = "%Y-%m-%d"
datey.x_labels = [
    datetime(2013, 1, 1),
    datetime(2013, 2, 1),
    datetime(2013, 3, 1)
]
datey.render()
```

Pies

Basic

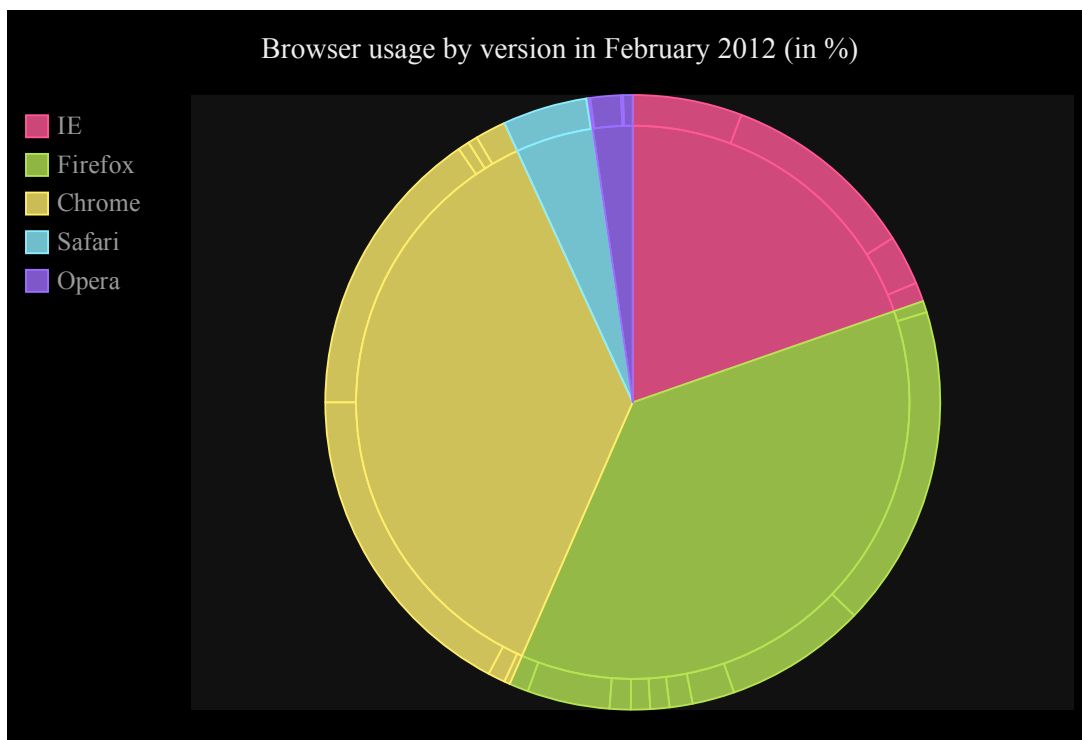
Simple pie:



```
pie_chart = pygal.Pie()
pie_chart.title = 'Browser usage in February 2012 (in %)'
pie_chart.add('IE', 19.5)
pie_chart.add('Firefox', 36.6)
pie_chart.add('Chrome', 36.3)
pie_chart.add('Safari', 4.5)
pie_chart.add('Opera', 2.3)
pie_chart.render()
```

Multi-series pie

Same pie but divided in sub category:

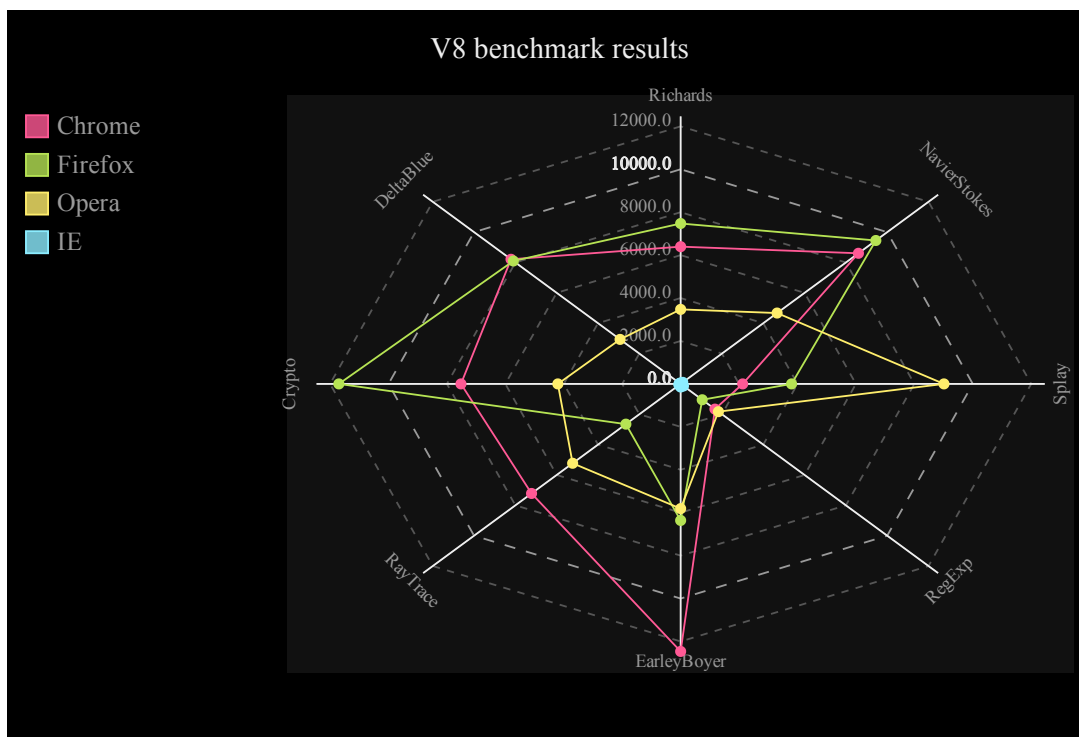


```
multiple_chart = pygal.Pie()
multiple_chart.title = 'Browser usage by version in February 2012 (in %)'
multiple_chart.add('IE', [5.7, 10.2, 2.6, 1])
multiple_chart.add('Firefox', [.6, 16.8, 7.4, 2.2, 1.2, 1])
multiple_chart.add('Chrome', [.3, .9, 17.1, 15.3, .6, .5])
multiple_chart.add('Safari', [4.4, .1])
multiple_chart.add('Opera', [.1, 1.6, .1, .5])
multiple_chart.render()
```

Radar charts

Basic

Simple Kiviat diagram:



```

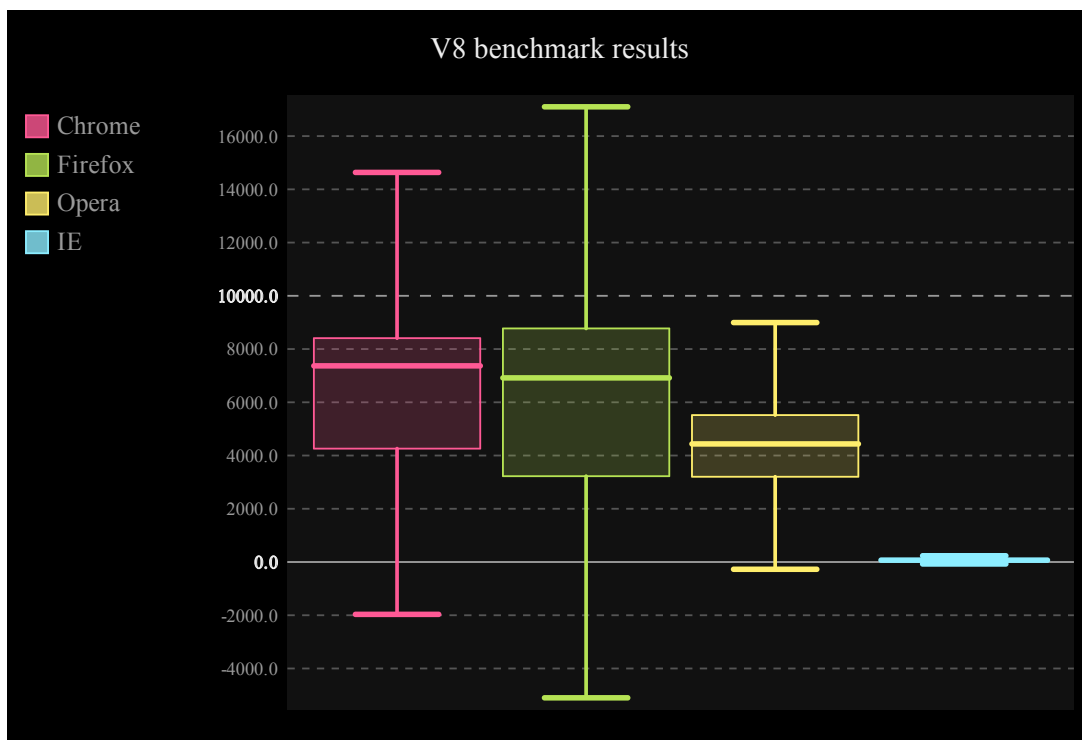
radar_chart = pygal.Radar()
radar_chart.title = 'V8 benchmark results'
radar_chart.x_labels = ['Richards', 'DeltaBlue', 'Crypto', 'NavierStokes', 'Splay', 'RegExp', 'EarleyBoyer', 'RayTrace']
radar_chart.add('Chrome', [6395, 8212, 7520, 7218, 12464, 18, 18, 18])
radar_chart.add('Firefox', [7473, 8099, 11700, 2651, 6361, 18, 18, 18])
radar_chart.add('Opera', [3472, 2933, 4203, 5229, 5810, 18, 18, 18])
radar_chart.add('IE', [43, 41, 59, 79, 144, 136, 34, 102])
radar_chart.render()

```

Box plot

Basic

Here's some whiskers:

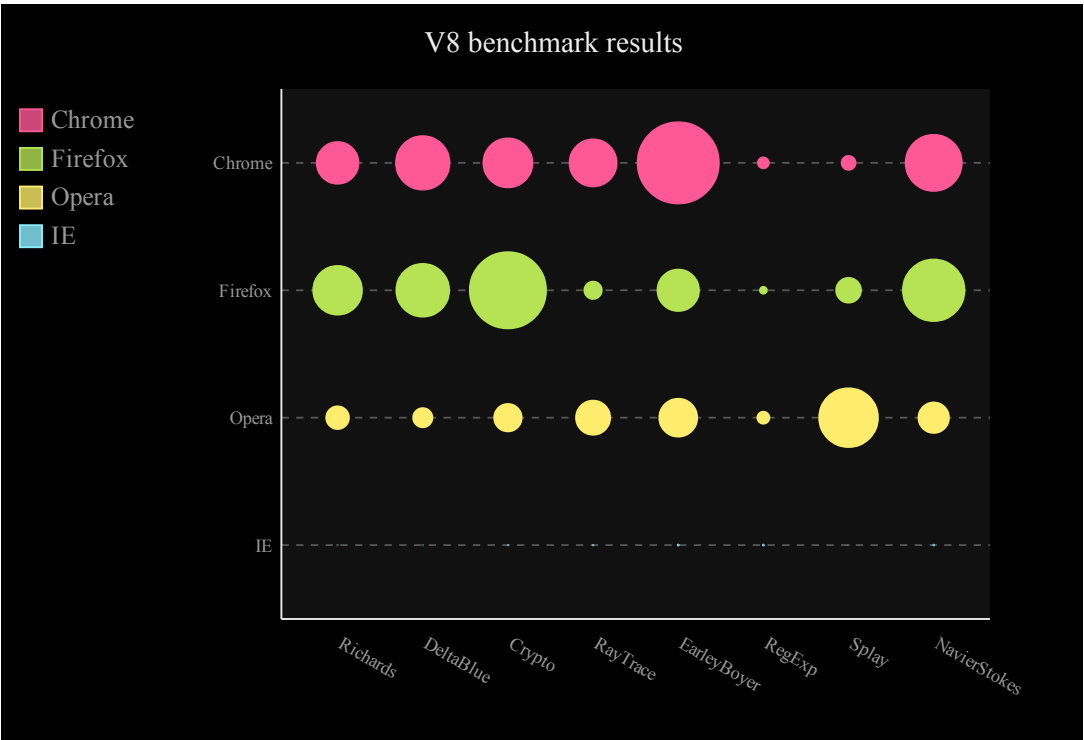


```
box_plot = pygal.Box()
box_plot.title = 'V8 benchmark results'
box_plot.add('Chrome', [6395, 8212, 7520, 7218, 12464, 16600])
box_plot.add('Firefox', [7473, 8099, 11700, 2651, 6361, 10000])
box_plot.add('Opera', [3472, 2933, 4203, 5229, 5810, 18280])
box_plot.add('IE', [43, 41, 59, 79, 144, 136, 34, 102])
box_plot.render()
```

Dot charts

Basic

Punch card like chart:



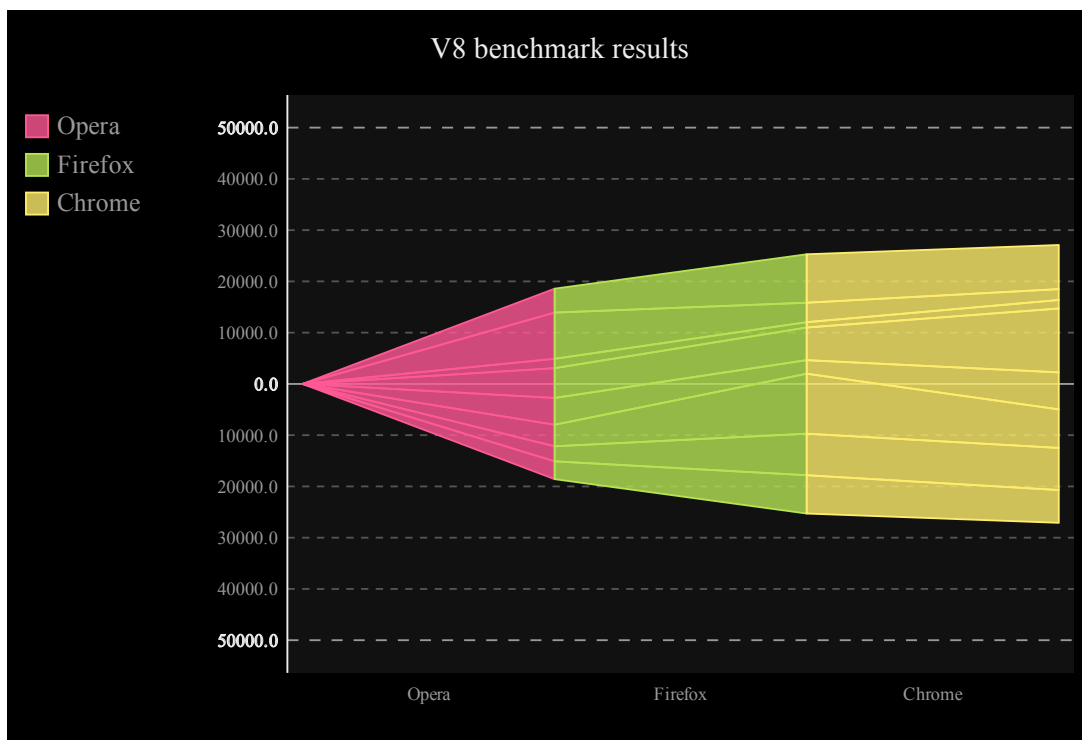
```
dot_chart = pygal.Dot(x_label_rotation=30)
dot_chart.title = 'V8 benchmark results'
dot_chart.x_labels = ['Richards', 'DeltaBlue', 'Crypto', 'RayTrace', 'EarleyBoyer', 'RegExp', 'Splay', 'NavierStokes']
dot_chart.add('Chrome', [6395, 8212, 7520, 7218, 12464, 16, 10, 10])
dot_chart.add('Firefox', [7473, 8099, 11700, 2651, 6361, 1, 1, 1])
dot_chart.add('Opera', [772, 2933, 4203, 5229, 5810, 1828, 1828, 1828])
dot_chart.add('IE', [45, 59, 79, 144, 136, 34, 102])
dot_chart.render()
```

Fork me on GitHub

Funnel charts

Basic

Funnel chart:

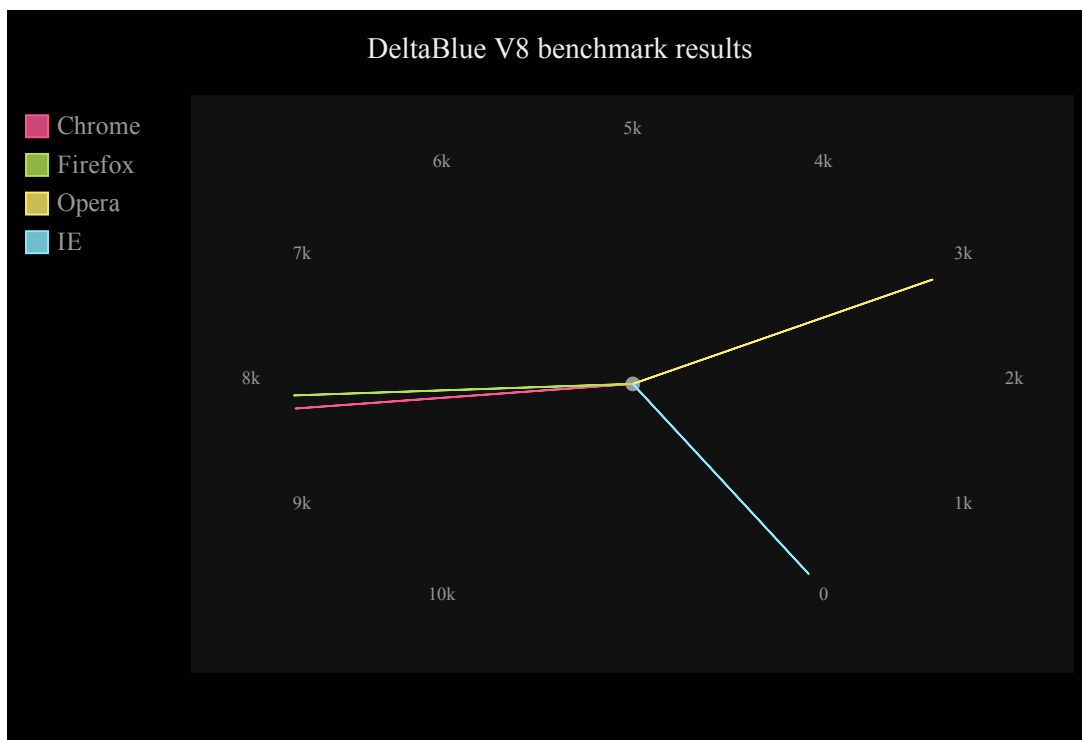


```
funnel_chart = pygal.Funnel()
funnel_chart.title = 'V8 benchmark results'
funnel_chart.x_labels = ['Richards', 'DeltaBlue', 'Crypto']
funnel_chart.add('Opera', [3472, 2933, 4203, 5229, 5810, 11700])
funnel_chart.add('Firefox', [7473, 8099, 11700, 2651, 6361, 11700])
funnel_chart.add('Chrome', [6395, 8212, 7520, 7218, 12464, 11700])
funnel_chart.render()
```

Gauge charts

Basic

Simple gauge chart:

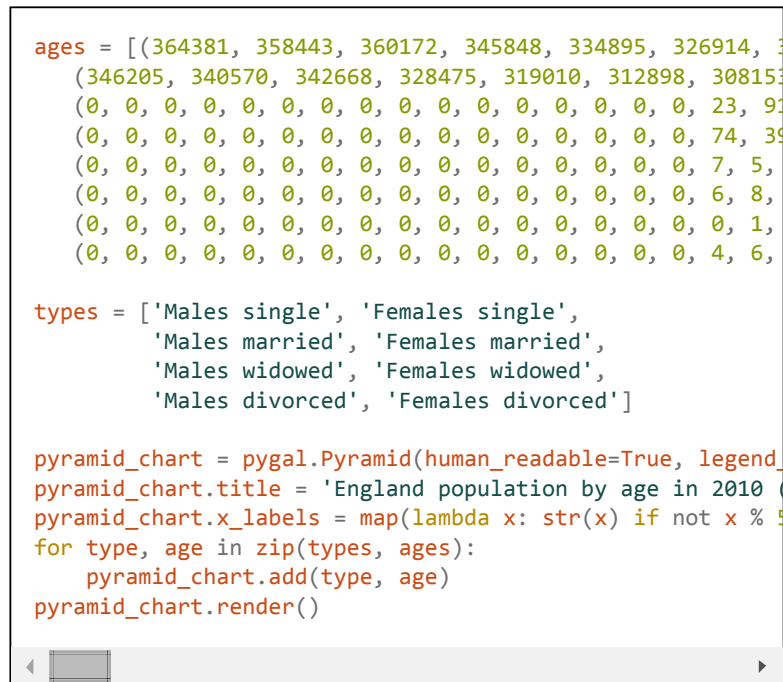


```
gauge_chart = pygal.Gauge(human_readable=True)
gauge_chart.title = 'DeltaBlue V8 benchmark results'
gauge_chart.x_labels = ['Richards', 'DeltaBlue', 'Crypto']
gauge_chart.range = [0, 10000]
gauge_chart.add('Chrome', 8212)
gauge_chart.add('Firefox', 8099)
gauge_chart.add('Opera', 2933)
gauge_chart.add('IE', 41)
gauge_chart.render()
```

Pyramid charts

Basic

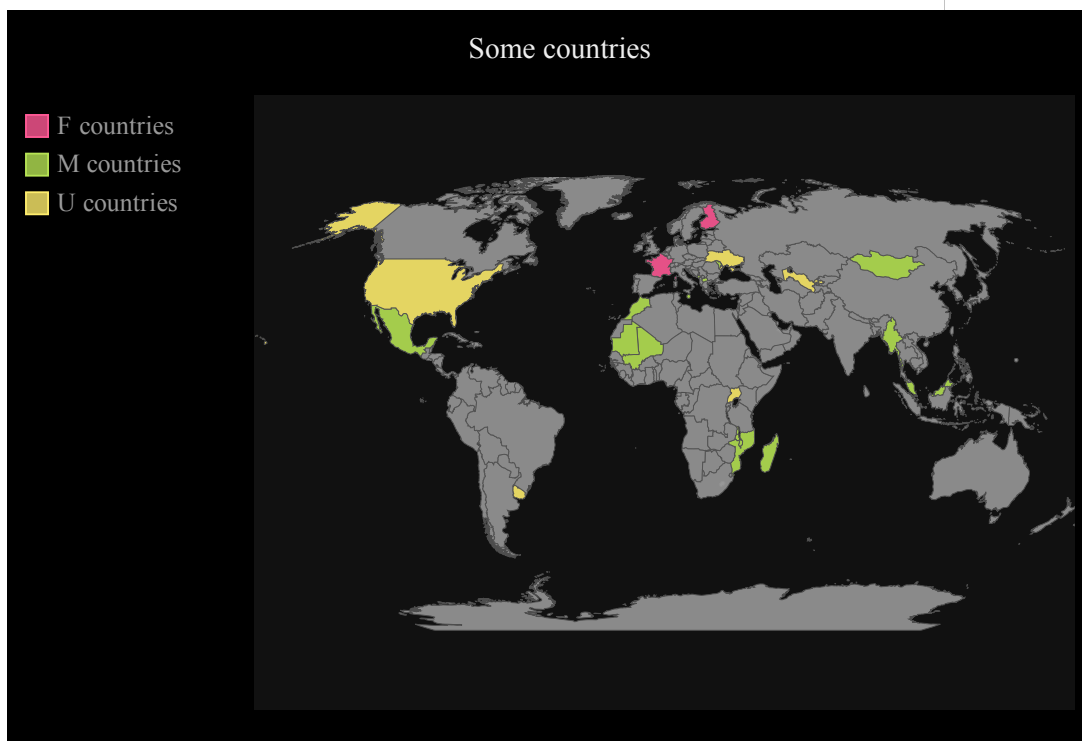
Population pyramid:



http://pygal.org/chart_types/

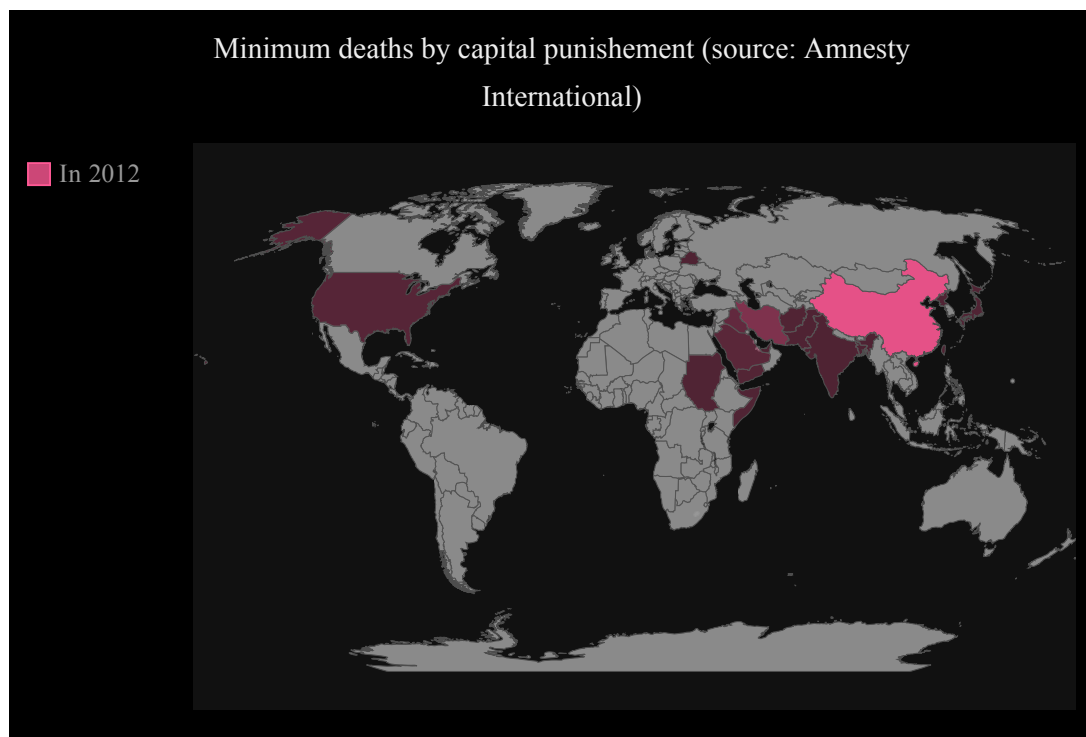
Basic

Highlight some countries:



```
worldmap_chart = pygal.Worldmap()
worldmap_chart.title = 'Some countries'
worldmap_chart.add('F countries', ['fr', 'fi'])
worldmap_chart.add('M countries', ['ma', 'mc', 'md', 'me',
                                   'mk', 'ml', 'mm', 'mn',
                                   'mr', 'mt', 'mu', 'mv',
                                   'mx', 'my', 'mz'])
worldmap_chart.add('U countries', ['ua', 'ug', 'us', 'uy'])
worldmap_chart.render()
```

You can also specify an number for a country:



```
worldmap_chart = pygal.Worldmap()
worldmap_chart.title = 'Minimum deaths by capital punishment'
worldmap_chart.add('In 2012', {
    'af': 14,
    'bd': 1,
    'by': 3,
    'cn': 1000,
    'gm': 9,
    'in': 1,
    'ir': 314,
    'iq': 129,
    'jp': 7,
    'kp': 6,
    'pk': 1,
    'ps': 6,
    'sa': 79,
    'so': 6,
    'sd': 5,
    'tw': 6,
    'ae': 1,
    'us': 43,
    'ye': 28
})
worldmap_chart.render()
```

The following countries are supported:

- **ad:** Andorra
- **ae:** United Arab Emirates
- **af:** Afghanistan
- **al:** Albania
- **am:** Armenia
- **ao:** Angola
- **aq:** Antarctica
- **ar:** Argentina
- **at:** Austria
- **au:** Australia

- [az](#): Azerbaijan
- [ba](#): Bosnia and Herzegovina
- [bd](#): Bangladesh
- [be](#): Belgium
- [bf](#): Burkina Faso
- [bg](#): Bulgaria
- [bh](#): Bahrain
- [bi](#): Burundi
- [bj](#): Benin
- [bn](#): Brunei Darussalam
- [bo](#): Bolivia, Plurinational State of
- [br](#): Brazil
- [bt](#): Bhutan
- [bw](#): Botswana
- [by](#): Belarus
- [bz](#): Belize
- [ca](#): Canada
- [cd](#): Congo, the Democratic Republic of the
- [cf](#): Central African Republic
- [cg](#): Congo
- [ch](#): Switzerland
- [ci](#): Cote d'Ivoire
- [cl](#): Chile
- [cm](#): Cameroon
- [cn](#): China
- [co](#): Colombia
- [cr](#): Costa Rica
- [cu](#): Cuba
- [cv](#): Cape Verde
- [cy](#): Cyprus
- [cz](#): Czech Republic
- [de](#): Germany
- [dj](#): Djibouti
- [dk](#): Denmark
- [do](#): Dominican Republic
- [dz](#): Algeria
- [ec](#): Ecuador
- [ee](#): Estonia
- [eg](#): Egypt
- [eh](#): Western Sahara
- [er](#): Eritrea
- [es](#): Spain
- [et](#): Ethiopia
- [fi](#): Finland
- [fr](#): France
- [ga](#): Gabon
- [gb](#): United Kingdom
- [ge](#): Georgia
- [gf](#): French Guiana
- [gh](#): Ghana
- [gl](#): Greenland
- [gm](#): Gambia
- [gn](#): Guinea
- [gq](#): Equatorial Guinea
- [gr](#): Greece
- [gt](#): Guatemala

- [gu](#): Guam
- [gw](#): Guinea-Bissau
- [gy](#): Guyana
- [hk](#): Hong Kong
- [hn](#): Honduras
- [hr](#): Croatia
- [ht](#): Haiti
- [hu](#): Hungary
- [id](#): Indonesia
- [ie](#): Ireland
- [il](#): Israel
- [in](#): India
- [iq](#): Iraq
- [ir](#): Iran, Islamic Republic of
- [is](#): Iceland
- [it](#): Italy
- [jm](#): Jamaica
- [jo](#): Jordan
- [jp](#): Japan
- [ke](#): Kenya
- [kg](#): Kyrgyzstan
- [kh](#): Cambodia
- [kp](#): Korea, Democratic People's Republic of
- [kr](#): Korea, Republic of
- [kw](#): Kuwait
- [kz](#): Kazakhstan
- [la](#): Lao People's Democratic Republic
- [lb](#): Lebanon
- [li](#): Liechtenstein
- [lk](#): Sri Lanka
- [lr](#): Liberia
- [ls](#): Lesotho
- [lt](#): Lithuania
- [lu](#): Luxembourg
- [lv](#): Latvia
- [ly](#): Libyan Arab Jamahiriya
- [ma](#): Morocco
- [mc](#): Monaco
- [md](#): Moldova, Republic of
- [me](#): Montenegro
- [mg](#): Madagascar
- [mk](#): Macedonia, the former Yugoslav Republic of
- [ml](#): Mali
- [mm](#): Myanmar
- [mn](#): Mongolia
- [mo](#): Macao
- [mr](#): Mauritania
- [mt](#): Malta
- [mu](#): Mauritius
- [mv](#): Maldives
- [mw](#): Malawi
- [mx](#): Mexico
- [my](#): Malaysia
- [mz](#): Mozambique
- [na](#): Namibia
- [ne](#): Niger

- [ng](#): Nigeria
- [ni](#): Nicaragua
- [nl](#): Netherlands
- [no](#): Norway
- [np](#): Nepal
- [nz](#): New Zealand
- [om](#): Oman
- [pa](#): Panama
- [pe](#): Peru
- [pg](#): Papua New Guinea
- [ph](#): Philippines
- [pk](#): Pakistan
- [pl](#): Poland
- [pr](#): Puerto Rico
- [ps](#): Palestine, State of
- [pt](#): Portugal
- [py](#): Paraguay
- [re](#): Reunion
- [ro](#): Romania
- [rs](#): Serbia
- [ru](#): Russian Federation
- [rw](#): Rwanda
- [sa](#): Saudi Arabia
- [sc](#): Seychelles
- [sd](#): Sudan
- [se](#): Sweden
- [sg](#): Singapore
- [sh](#): Saint Helena, Ascension and Tristan da Cunha
- [si](#): Slovenia
- [sk](#): Slovakia
- [sl](#): Sierra Leone
- [sm](#): San Marino
- [sn](#): Senegal
- [so](#): Somalia
- [sr](#): Suriname
- [st](#): Sao Tome and Principe
- [sv](#): El Salvador
- [sy](#): Syrian Arab Republic
- [sz](#): Swaziland
- [td](#): Chad
- [tg](#): Togo
- [th](#): Thailand
- [tj](#): Tajikistan
- [tl](#): Timor-Leste
- [tm](#): Turkmenistan
- [tn](#): Tunisia
- [tr](#): Turkey
- [tw](#): Taiwan, Province of China
- [tz](#): Tanzania, United Republic of
- [ua](#): Ukraine
- [ug](#): Uganda
- [us](#): United States
- [uy](#): Uruguay
- [uz](#): Uzbekistan
- [va](#): Holy See (Vatican City State)

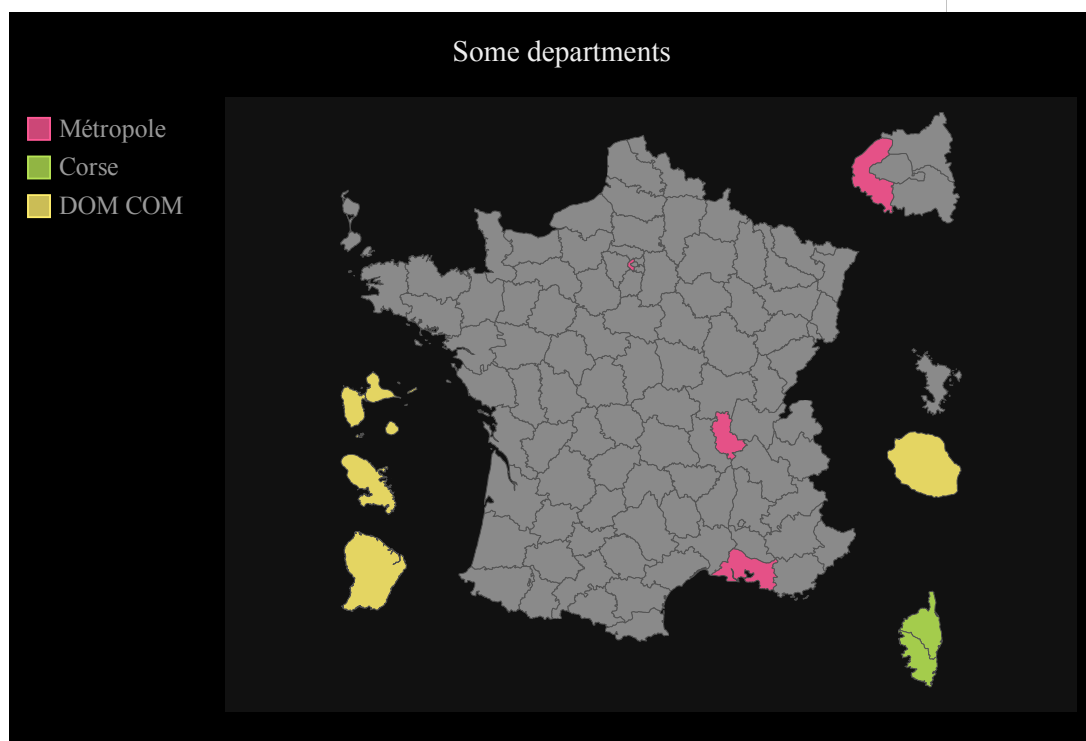
- **ve**: Venezuela, Bolivarian Republic of
- **vn**: Viet Nam
- **ye**: Yemen
- **yt**: Mayotte
- **za**: South Africa
- **zm**: Zambia
- **zw**: Zimbabwe

Country charts

As of now, only France is available. As other country are implemented, this will be externalized in other packages. (Please submit pull requests :))

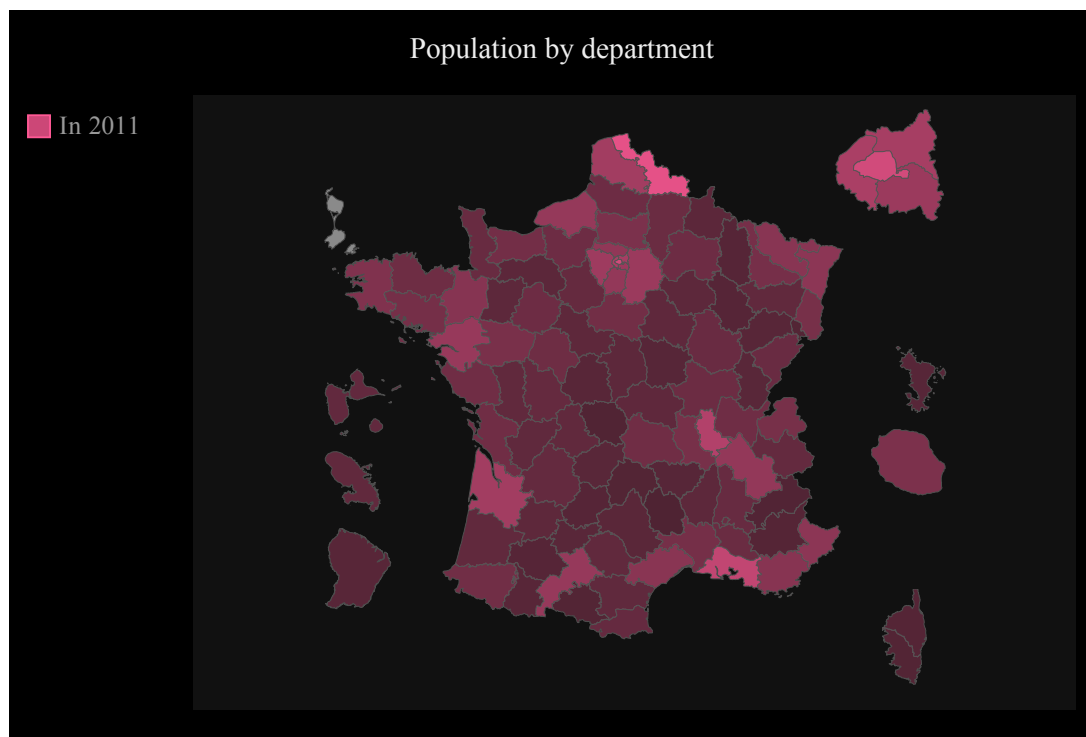
French map

Highlight some departments:



```
fr_chart = pygal.FrenchMap_Departments()
fr_chart.title = 'Some departments'
fr_chart.add('Métropole', ['69', '92', '13'])
fr_chart.add('Corse', ['2A', '2B'])
fr_chart.add('DOM COM', ['971', '972', '973', '974'])
fr_chart.render()
```

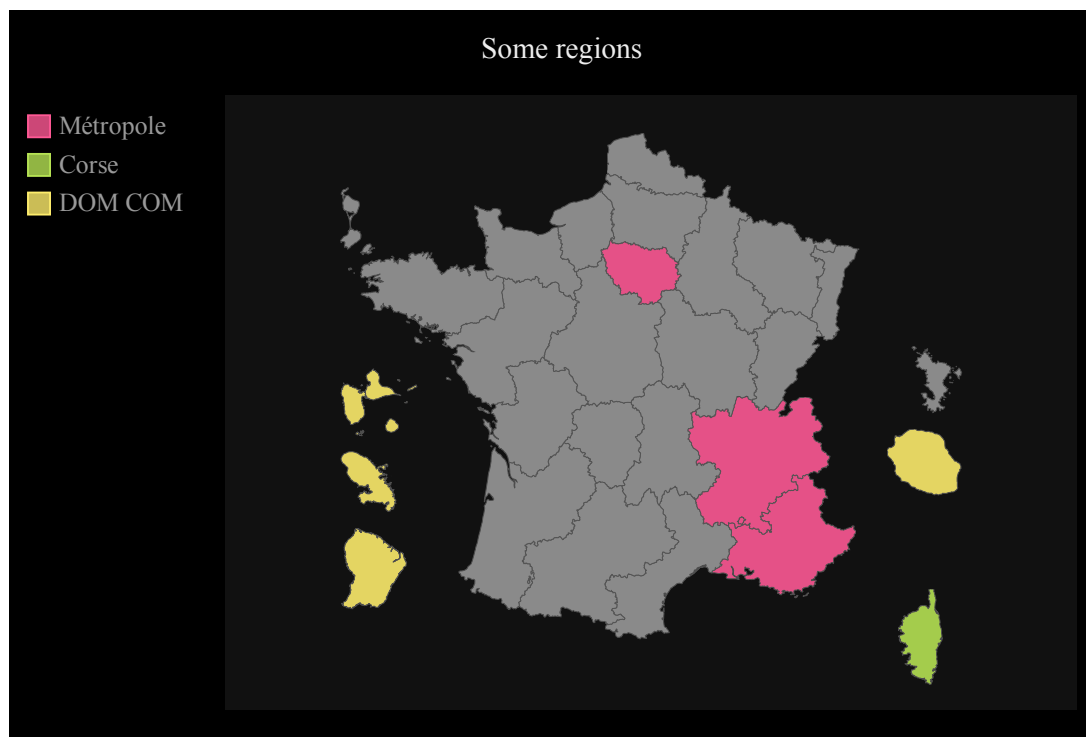
You can also specify an number for a department:



```
fr_chart = pygal.FrenchMap_Departments(human_readable=True)
fr_chart.title = 'Population by department'
fr_chart.add('In 2011', {
    '01': 603827,
    '02': 541302,
    '03': 342729,
    '04': 160959,
    '05': 138605,
    '06': 1081244,
    '07': 317277,
    '08': 283110,
    '09': 152286,
    '10': 303997,
    '11': 359967,
    '12': 275813,
    '13': 1975896,
    '14': 685262,
    '15': 147577,
    '16': 352705,
    '17': 625682,
    '18': 311694,
    '19': 242454,
    '2A': 145846,
    '2B': 168640,
    '21': 525931,
    '22': 594375,
    '23': 122560,
    '24': 415168,
    '25': 529103,
    '26': 487993,
    '27': 588111,
    '28': 430416,
    '29': 899870,
    '30': 718357,
    '31': 1260226,
    '32': 188893,
    '33': 1463662,
    '34': 1062036,
    '35': 996439,
    '36': 230175,
    '37': 593683,
    '38': 1215212,
    '39': 261294,
    '40': 387929,
```

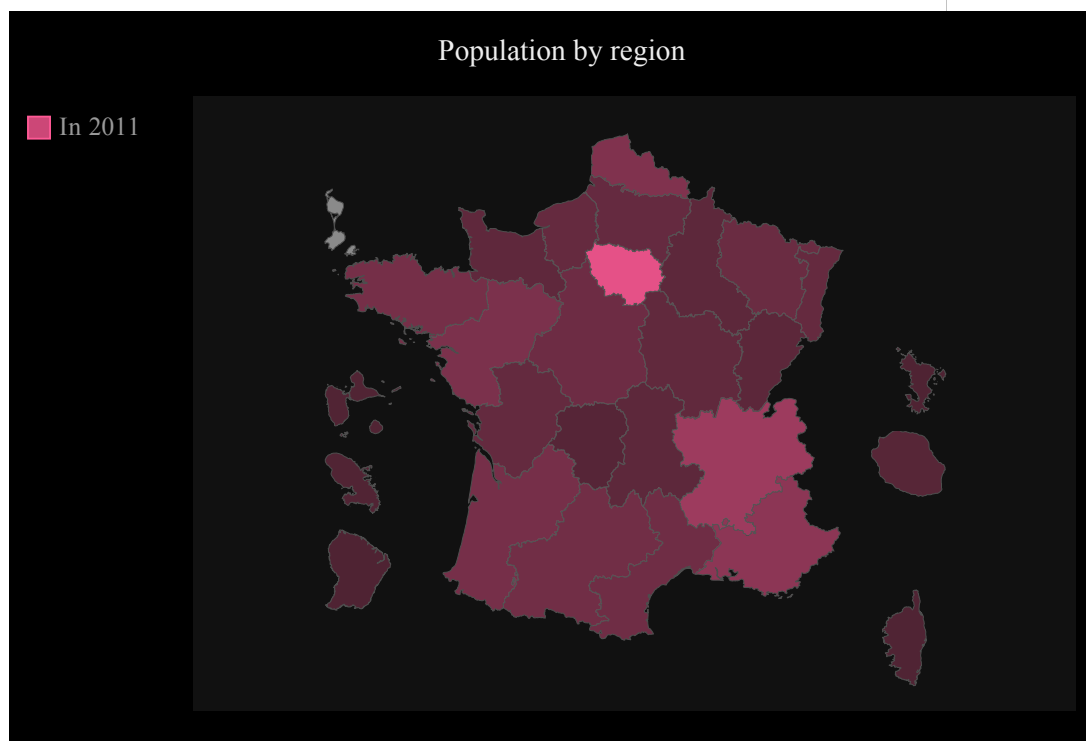
```
'41': 331280,  
'42': 749053,  
'43': 224907,  
'44': 1296364,  
'45': 659587,  
'46': 174754,  
'47': 330866,  
'48': 77156,  
'49': 790343,  
'50': 499531,  
'51': 566571,  
'52': 182375,  
'53': 307031,  
'54': 733124,  
'55': 193557,  
'56': 727083,  
'57': 1045146,  
'58': 218341,  
'59': 2579208,  
'60': 805642,  
'61': 290891,  
'62': 1462807,  
'63': 635469,  
'64': 656608,  
'65': 229228,  
'66': 452530,  
'67': 1099269,  
'68': 753056,  
'69': 1744236,  
'70': 239695,  
'71': 555999,  
'72': 565718,  
'73': 418949,  
'74': 746994,  
'75': 2249975,  
'76': 1251282,  
'77': 1338427,  
'78': 1413635,  
'79': 370939,  
'80': 571211,  
'81': 377675,  
'82': 244545,  
'83': 1012735,  
'84': 546630,  
'85': 641657,  
'86': 428447,  
'87': 376058,  
'88': 378830,  
'89': 342463,  
'90': 143348,  
'91': 1225191,  
'92': 1581628,  
'93': 1529928,  
'94': 1333702,  
'95': 1180365,  
'971': 404635,  
'972': 392291,  
'973': 237549,  
'974': 828581,  
'976': 212645  
})  
fr_chart.render()
```

You can do the same with regions:



```
fr_chart = pygal.FrenchMap_Regions()
fr_chart.title = 'Some regions'
fr_chart.add('Métropole', ['82', '11', '93'])
fr_chart.add('Corse', ['94'])
fr_chart.add('DOM COM', ['01', '02', '03', '04'])
fr_chart.render()
```

You can also specify a number for a region and use a department to region aggregation:



```
from pygal.graph.frenchmap import aggregate_regions
fr_chart = pygal.FrenchMap_Regions(human_readable=True)
fr_chart.title = 'Population by region'
fr_chart.add('In 2011', aggregate_regions({
```

```
'01': 603827,  
'02': 541302,  
'03': 342729,  
'04': 160959,  
'05': 138605,  
'06': 1081244,  
'07': 317277,  
'08': 283110,  
'09': 152286,  
'10': 303997,  
'11': 359967,  
'12': 275813,  
'13': 1975896,  
'14': 685262,  
'15': 147577,  
'16': 352705,  
'17': 625682,  
'18': 311694,  
'19': 242454,  
'2A': 145846,  
'2B': 168640,  
'21': 525931,  
'22': 594375,  
'23': 122560,  
'24': 415168,  
'25': 529103,  
'26': 487993,  
'27': 588111,  
'28': 430416,  
'29': 899870,  
'30': 718357,  
'31': 1260226,  
'32': 188893,  
'33': 1463662,  
'34': 1062036,  
'35': 996439,  
'36': 230175,  
'37': 593683,  
'38': 1215212,  
'39': 261294,  
'40': 387929,  
'41': 331280,  
'42': 749053,  
'43': 224907,  
'44': 1296364,  
'45': 659587,  
'46': 174754,  
'47': 330866,  
'48': 77156,  
'49': 790343,  
'50': 499531,  
'51': 566571,  
'52': 182375,  
'53': 307031,  
'54': 733124,  
'55': 193557,  
'56': 727083,  
'57': 1045146,  
'58': 218341,  
'59': 2579208,  
'60': 805642,  
'61': 290891,  
'62': 1462807,  
'63': 635469,  
'64': 656608,  
'65': 229228,  
'66': 452530,  
'67': 1099269,  
'68': 753056,  
'69': 1744236,  
'70': 239695,
```

```
'71': 555999,  
'72': 565718,  
'73': 418949,  
'74': 746994,  
'75': 2249975,  
'76': 1251282,  
'77': 1338427,  
'78': 1413635,  
'79': 370939,  
'80': 571211,  
'81': 377675,  
'82': 244545,  
'83': 1012735,  
'84': 546630,  
'85': 641657,  
'86': 428447,  
'87': 376058,  
'88': 378830,  
'89': 342463,  
'90': 143348,  
'91': 1225191,  
'92': 1581628,  
'93': 1529928,  
'94': 1333702,  
'95': 1180365,  
'971': 404635,  
'972': 392291,  
'973': 237549,  
'974': 828581,  
'976': 212645  
}))  
fr_chart.render()
```

Next: [Styles](#)